Building Your Personal AI Assistant

Telegram + n8n + Llama + ChromaDB + Gmail

AI Agents Classroom Series • 2 Hours • thecoachdinesh.com

Telegram	n8n Router	Llama 3.1	ChromaDB	Gmail
Chat UI	Routes intent	Local LLM	Vector memory	Email action

⏱️ **Duration:** 2 hours — Hour 1: setup + basic bot | Hour 2: memory + actions

PART 1 — Install Ollama and Run Llama Locally (15 minutes)

What is Ollama?

Ollama is a tool that lets you run AI models like Llama directly on your laptop. No internet needed, no API costs, completely private.

Step 1 — Install Ollama

Mac — open Terminal and run:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Windows — download the installer:

```
https://ollama.com/download
Install it like any normal app, then open Command Prompt.
```

Step 2 — Download the Llama model

```
ollama pull llama3.2:3b
```

> 💡 **Why 3B?** 3B model is only 2GB and runs on any laptop with 8GB RAM. Fast enough for classroom demos!

Step 3 — Test Ollama is working

```
ollama run llama3.2:3b "Say hello in one sentence"
```

> ✅ **Success:** Ollama runs at http://localhost:11434 — n8n will call this URL. From inside Docker use host.docker.internal:11434

PART 2 — Set Up ChromaDB (Vector Memory) (10 minutes)

What is ChromaDB?

ChromaDB stores text as numbers (embeddings). When you search, it finds the most similar text by meaning — not just keywords. This gives your bot long-term memory!

Step 1 — Run ChromaDB in Docker

```
docker run -d --name chromadb -p 8000:8000 chromadb/chroma
```

Step 2 — Test ChromaDB is working

Open browser and go to:

```
http://localhost:8000/api/v2/heartbeat
```

You should see: {"nanosecond heartbeat": 12345678}

Step 3 — Create the memories collection

```
curl -X POST http://localhost:8000/api/v2/collections \
  -H "Content-Type: application/json" \
  -d '{"name": "memories"}'
```

✅ **Success:** ChromaDB is at http://localhost:8000 — use host.docker.internal:8000 from inside n8n Docker

PART 3 — Build the Telegram Bot in n8n (30 minutes)

Step 1 — Create new workflow in n8n

1	**Open n8n** Go to http://localhost:5678 in Chrome or Firefox
2	**Create new workflow** Click the + icon → New Workflow
3	**Name it** Click "My workflow" at top → type "Personal AI Assistant"

Step 2 — Add Telegram Trigger node

1	**Click + on canvas** Search for "Telegram Trigger" and select it
2	**Set up credential** Click "Create new credential" → paste your Bot Token from @BotFather
3	**Set updates to "message"** In the updates field select "message"

Step 3 — Add Code node (Extract & Route)

1	**Click + after Telegram Trigger** Search for "Code" and select it
2	**Set mode** Set Mode to "Run Once for Each Item"
3	**Paste this code** Replace ALL existing code with the block below:

```
const msg = $json.message;
const text = msg?.text || '';
const chatId = msg?.chat?.id;
const name = msg?.from?.first_name || 'Student';

let intent = 'question';
if (text.toLowerCase().startsWith('remember')) intent = 'remember';
else if (text.toLowerCase().includes('search') ||
         text.toLowerCase().includes('find')) intent = 'search';
else if (text.toLowerCase().includes('email') ||
         text.toLowerCase().includes('send')) intent = 'email';

return [{ json: { text, chatId, name, intent } }];
```

Step 4 — Add Switch node (Router)

1	**Click + after Code node** Search for "Switch" and select it
2	**Set Mode to "Rules"**
3	**Add rules based on {{ $json.intent }}**

Rule 1: {{ $json.intent }} equals remember → Output 0

Rule 2: {{ $json.intent }} equals search → Output 1

Rule 3: {{ $json.intent }} equals email → Output 2

Rule 4: Fallback (else) → Output 3

PART 4 — Connect Llama (Local AI) (20 minutes)

Step 1 — Add HTTP Request node for Llama

Connect from Switch Output 3 (fallback — general questions):

1	**Add HTTP Request node** Click + from Switch fallback output
2	**Method** POST
3	**URL** http://host.docker.internal:11434/api/generate

⚠️ **Important!** Use host.docker.internal — NOT localhost. n8n runs inside Docker and needs this special address to reach Ollama on your Mac/PC

Step 2 — Set the JSON body

```
{
  "model": "llama3.2:3b",
  "prompt": "{{ $json.text }}",
  "stream": false
}
```

Step 3 — Add Code node to extract answer

```
const answer = $input.first().json.response;
const chatId = $('Code').first().json.chatId;
return [{ json: { answer, chatId } }];
```

Step 4 — Send reply to Telegram

1	**Add HTTP Request node** Method: POST
2	**URL** https://api.telegram.org/botYOUR_BOT_TOKEN/sendMessage
3	**JSON Body**

```
{
  "chat_id": "{{ $json.chatId }}",
  "text": "{{ $json.answer }}"
}
```

✅ **Test checkpoint!** Save and click "Test Workflow". Send any message to your Telegram bot — it should reply using Llama running on your laptop!

PART 5 — Connect ChromaDB Memory (25 minutes)

Save memory — connect from Switch Output 0 (remember intent)

1	**Add HTTP Request node** Connect from Switch Output 0
2	**Method** POST
3	**URL** http://host.docker.internal:8000/api/v2/collections/memories/add

```
{
  "documents": ["{{ $json.text }}"],
  "ids": ["mem_{{ $now.toMillis() }}"],
  "metadatas": [{"saved_by": "{{ $json.name }}"}]
}
```

4	**Add Telegram reply** Send: Got it! Saved to your memory. You can search for it later!

Search memory — connect from Switch Output 1 (search intent)

1	**Add HTTP Request node** Connect from Switch Output 1
2	**Method** POST
3	**URL** http://host.docker.internal:8000/api/v2/collections/memories/query

```
{
  "query_texts": ["{{ $json.text }}"],
  "n_results": 3
}
```

4	**Add Code node** Extract results and pass to Llama:

```
const results = $json.documents?.[0] || [];
const context = results.length > 0 ? results.join('\n') : 'No memories found.';
const question = $('Code').first().json.text;
const chatId = $('Code').first().json.chatId;
return [{ json: { context, question, chatId } }];
```

5	**Add Llama HTTP Request** Same URL as before but include context in prompt:

```
{
  "model": "llama3.2:3b",
  "prompt": "You are a personal assistant. Based on these memories:\
n{{ $json.context }}\n\nAnswer this question: {{ $json.question }}",
  "stream": false
}
```

PART 6 — Add Gmail Email Action (15 minutes)

Connect from Switch Output 2 (email intent)

1	**Add Gmail node** Connect from Switch Output 2
2	**Set up credential** Use your Google OAuth Client ID + Secret (same as before)
3	**Operation** Send
4	**To field** ={{ $json.text.match(/to\s+([\w.@]+)/i)?.[1] || 'your@email.com' }}
5	**Subject** Message from your AI Assistant
6	**Message** ={{ $json.text }}
7	**Add Telegram reply** Send: Email sent successfully! ✅

> ✅ **Final test:** Try: "Send email to professor@college.edu saying I need an extension" — it should send the email and reply on Telegram!

Quick Reference — All URLs and Commands

Service	URL	Notes
n8n	`http://localhost:5678`	Your workflow builder
Ollama / Llama	`http://host.docker.internal:11434`	From inside Docker n8n
ChromaDB	`http://host.docker.internal:8000`	From inside Docker n8n
Telegram Bot API	`https://api.telegram.org/botTOKEN/`	Replace TOKEN with yours
Groq (backup LLM)	`https://api.groq.com/openai/v1/chat/completions`	If Ollama is too slow
Gmail OAuth Redirect	`http://localhost:5678/rest/oauth2-credential/callback`	For Google Cloud Console

Troubleshooting

Problem	Cause	Fix
Ollama not responding	host.docker.internal not resolving	On Windows try 172.17.0.1 instead
ChromaDB 404 error	Collection not created	Run the curl command in Part 2 Step 3 first
Llama too slow	Large model / low RAM	Use llama3.2:3b — only 2GB, much faster
Telegram not responding	Workflow not published	Click Publish — saving alone does not activate triggers
Gmail OAuth blocked	Not added as test user	Google Cloud → OAuth Consent → Audience → add your Gmail
n8n cant reach Ollama	Using localhost in Docker	Use host.docker.internal NOT localhost in node URLs

You just built a personal AI assistant with local LLM + vector memory + email actions!